

MÁSTER EN BIOINFORMÁTICA Y BIOESTADÍSTICA

PEC 2

Análisis de datos en R: programación, SQL y simulación estadística

Software para el Análisis de Datos

Autora: Dra. Marta Barea Sepúlveda

Fecha: 22 de abril de 2026

Índice

1. Introducción	1
2. Sección 1: Fundamentos de programación y BBDD	2
2.1. Ejercicio 1. Análisis de secuencias de ADN en una matriz	2
2.1.1. Enunciado	2
2.1.2. Resolución	2
2.2. Ejercicio 2. Cálculo del índice de normalización de expresión.	4
2.2.1. Enunciado	4
2.2.2. Resolución	5
2.3. Ejercicio 3. Análisis farmacocinético con SQL en R	7
2.3.1. Enunciado	7
2.3.2. Resolución	8
3. Sección 2: Probabilidad y simulaciones	13
3.1. Ejercicio 4. Tiempo de eliminación de un fármaco en sangre	13
3.1.1. Enunciado	13
3.1.2. Resolución	13
3.2. Ejercicio 5. Respuesta a un tratamiento analgésico en dolor crónico.	18
3.2.1. Enunciado	18
3.2.2. Resolución	19

1. Introducción

La presente memoria recoge el desarrollo de la **PEC2** de la asignatura *Software para el Análisis de Datos*, perteneciente al **Máster en Bioinformática y Bioestadística** (UOC-UB). El objetivo principal es afianzar los conocimientos adquiridos en los **Laboratorios 3 y 4**, centrados en el uso del lenguaje **R** para el análisis y la simulación de datos.

En el **Laboratorio 3** se trabajan los fundamentos de programación en R, incluyendo el uso de **estructuras condicionales, bucles y funciones**, así como una primera aproximación al manejo de **bases de datos**. Por su parte, el **Laboratorio 4** introduce los conceptos básicos de **probabilidad y simulación**, abordando la generación de **números pseudoaleatorios** y la simulación de **distribuciones estadísticas**.

A lo largo de la memoria se aplican estos conceptos en la resolución de distintos ejercicios, reforzando competencias clave como la **programación en R**, el **análisis de datos** y la **modelización estadística**. El código relativo a los resultados presentados en esta memoria se encuentra disponible para su consulta en el **repositorio de GitHub** asociado al trabajo: [data-analysis-r](#).

2. Sección 1: Fundamentos de programación y BBDD

2.1. Ejercicio 1. Análisis de secuencias de ADN en una matriz

2.1.1. Enunciado

Una secuencia de ADN está formada por los caracteres "A", "C", "G" y "T". Supongamos que, además de la transcripción a ARN, queremos analizar varias secuencias almacenadas en forma de matriz. Para ello, se dispone de la siguiente matriz, donde cada fila representa una secuencia de ADN:

```
secuencias <- matrix(c(
  "GATTACA",
  "GCGTTA",
  "TTTACG",
  "ACGTACGT"
), ncol = 1)
```

Se pide:

- Recorrer la matriz utilizando un bucle y mostrar el resultado de cada iteración.
- A partir de la matriz `secuencias` definida en el enunciado, ¿cuántas filas contienen al menos un nucleótido "T"?
- Definir una función llamada `DNA_to_RNA` que convierta una cadena de ADN en ARN sustituyendo todas las "T" por "U", y mostrar el resultado para una secuencia de ejemplo, como "ATTGCTT". Para asegurar los datos de entrada, también se pide crear una función `validar_adn` que, dada una secuencia, valide que dicha secuencia sólo contenga los caracteres válidos: "A", "C", "G" y "T". De esta manera, si se detecta algún carácter distinto, se muestra un mensaje de error y no se procesa esa secuencia.
- Ejecutar la función `DNA_to_RNA`, pasando la matriz `secuencias` como parámetro y mostrar el resultado iteración a iteración.

2.1.2. Resolución

Se parte de la matriz `secuencias`, donde cada fila contiene una cadena de ADN. En este ejercicio, el objetivo es recorrer esas secuencias, analizarlas y transformarlas en ARN cuando corresponda:

- Para este apartado se usa un bucle `for`, que permite recorrer cada una de las filas de la matriz. El código es el siguiente:

```
# Recorrer la matriz y mostrar cada secuencia
for (i in 1:nrow(secuencias)) {
  cat("Fila", i, ":", secuencias[i, 1], "\n")
}
```

```
## Fila 1 : GATTACA
## Fila 2 : GCGTTA
## Fila 3 : TTTACG
## Fila 4 : ACGTACGT
```

Como se observa en la salida, el bucle recorre todas las filas de **secuencias** una a una y muestra su contenido junto con el número de fila. De este modo, se puede comprobar de forma clara que el recorrido de la matriz se está realizando correctamente.

- b) Para contar cuántas filas contienen al menos un nucleótido "T", se utiliza la función **grepl**, que permite buscar patrones dentro de cadenas. Se recorre cada fila de la matriz y se incrementa un contador cada vez que aparece una "T".

El código para este apartado es el siguiente:

```
# Contar filas que contienen al menos un 'T'
contador_T <- 0
for (i in 1:nrow(secuencias)) {
  if (grepl("T", secuencias[i, 1])) {
    contador_T <- contador_T + 1
  }
}
cat("Número de filas que contienen al menos un nucleótido 'T':", contador_T,
    ↵ "\n")
```

```
## Número de filas que contienen al menos un nucleótido 'T': 4
```

En este caso, el resultado indica que hay 4 filas con al menos un nucleótido "T". Esto significa que todas las secuencias de la matriz contienen, al menos una vez, dicho nucleótido.

- c) A continuación, se define la función **DNA_to_RNA**, que convierte una cadena de ADN en ARN sustituyendo todas las "T" por "U". Además, se crea la función **validar_adn** para comprobar que las secuencias de ADN solo contienen caracteres válidos.

El código para estas funciones es el siguiente:

```
# Función para validar la secuencia de ADN
validar_adn <- function(secuencia) {
  if (grepl("[ACGT]+$", secuencia)) {
    return(TRUE)
  } else {
    cat("Error: La secuencia contiene caracteres no válidos.\n")
    return(FALSE)
  }
}

# Función para convertir ADN a ARN
DNA_to_RNA <- function(adn) {
```

```
if (validar_adn(adn)) {  
  rna <- gsub("T", "U", adn)  
  return(rna)  
} else {  
  return(NULL)  
}  
}
```

En esta parte, la validación previa resulta importante porque evita procesar secuencias incorrectas y hace que la función sea más robusta. Así, antes de realizar la conversión, se comprueba que la cadena realmente corresponde a una secuencia de ADN válida.

- d) Finalmente, se ejecuta la función `DNA_to_RNA` sobre cada una de las secuencias de la matriz `secuencias` y se muestra el resultado en cada iteración:

```
# Ejecutar la función DNA_to_RNA sobre la matriz secuencias  
for (i in 1:nrow(secuencias)) {  
  adn <- secuencias[i, 1]  
  rna <- DNA_to_RNA(adn)  
  if (!is.null(rna)) {  
    cat("Fila", i, "ADN:", adn, "-> ARN:", rna, "\n")  
  }  
}
```

```
## Fila 1 ADN: GATTACA -> ARN: GAUUACA  
## Fila 2 ADN: GCGTTA -> ARN: GCGUUA  
## Fila 3 ADN: TTTACG -> ARN: UUUACG  
## Fila 4 ADN: ACGTACGT -> ARN: ACGUACGU
```

La salida muestra la conversión de cada secuencia de ADN a ARN, indicando tanto la cadena original como la resultante. En este caso, todas las secuencias son válidas, por lo que la transformación se realiza sin problemas en todos los casos y se obtiene el resultado esperado.

2.2. Ejercicio 2. Cálculo del índice de normalización de expresión.

2.2.1. Enunciado

Supongamos un experimento de **RNA-seq**, a partir del cual queremos normalizar la expresión de un gen respecto al total de lecturas de la muestra. Este ajuste de los datos se realiza para permitir que sean comparables.

Supongamos que se define el índice de normalización como:

$$indice = \frac{lecturas_gen}{lecturas_totales}$$

Hay que tener en cuenta que dichos valores han de ser **numéricos y positivos**.

Resolved las siguientes cuestiones:

a) Definir una función en R llamada `indice_normalizacion` que reciba dos parámetros:

- `lecturas_gen`: número de lecturas del gen
- `lecturas_totales`: número total de lecturas de la muestra

La función a definir, debe calcular y devolver el valor del índice, así como también realizar el control de errores que corresponda.

b) Comprobar el resultado para, por ejemplo, los siguientes parámetros:

- `lecturas_gen = 500`
- `lecturas_totales = 10000`

Además, realizar pruebas con distintos tipos de valores (numéricos, no numéricos, etc.).

c) ¿Cómo modificaríais el código para permitir al usuario introducir valores?

2.2.2. Resolución

a) Para definir la función `indice_normalizacion`, es necesario incluir un control de errores que verifique que los parámetros recibidos sean numéricos y positivos. Esta comprobación es importante, ya que el índice solo tiene sentido si se trabaja con valores válidos. Una vez superada esa validación, se calcula el índice de normalización y se devuelve el resultado.

Para ello, el código correspondiente a esta función es el siguiente:

```
indice_normalizacion <- function(lecturas_gen, lecturas_totales) {
  # Control de errores
  if (!is.numeric(lecturas_gen) || !is.numeric(lecturas_totales)) {
    stop("Error: Ambos parámetros deben ser numéricos.")
  }
  if (lecturas_gen < 0 || lecturas_totales <= 0) {
    stop("Error: Las lecturas deben ser positivas y el total debe ser mayor
      ↪ que cero.")
  }

  # Cálculo del índice
  indice <- lecturas_gen / lecturas_totales
  return(indice)
}
```

b) Para comprobar el resultado de la función anterior, se pueden hacer pruebas con los parámetros del enunciado y con otros valores. De este modo, no solo se verifica que el cálculo sea correcto en un caso válido, sino también que el control de errores funcione como debe ante entradas incorrectas.

El código para estas pruebas es el que se muestra a continuación:

```
# Prueba con valores válidos
lecturas_gen <- 500
lecturas_totales <- 10000
resultado <- indice_normalizacion(lecturas_gen, lecturas_totales)
cat("Índice de normalización:", resultado, "\n")
```

```
## Índice de normalización: 0.05
```

```
# Prueba con valores no numéricos
tryCatch({
  indice_normalizacion("cincuenta", 10000)
}, error = function(e) {
  cat(e$message, "\n")
})
```

```
## Error: Ambos parámetros deben ser numéricos.
```

```
# Prueba con valores negativos
tryCatch({
  indice_normalizacion(-500, 10000)
}, error = function(e) {
  cat(e$message, "\n")
})
```

```
## Error: Las lecturas deben ser positivas y el total debe ser mayor que cero.
```

```
# Prueba con total de lecturas igual a cero
tryCatch({
  indice_normalizacion(500, 0)
}, error = function(e) {
  cat(e$message, "\n")
})
```

```
## Error: Las lecturas deben ser positivas y el total debe ser mayor que cero.
```

A partir de los resultados, se comprueba que la función devuelve el índice correcto cuando los valores son válidos y que muestra mensajes de error en los casos no válidos. Por tanto, la función no solo calcula el índice, sino que además controla adecuadamente situaciones que podrían dar lugar a resultados erróneos.

- c) Para permitir la introducción de valores por parte del usuario, se puede utilizar la función `readline`, ya que solicita la entrada desde la consola. Después, se convierte la entrada a formato numérico y se llama a `indice_normalizacion` con los valores introducidos.

El código relativo a esta modificación es el siguiente:


```
# Solicitar al usuario que introduzca los valores
lecturas_gen <- as.numeric(readline(prompt = "Introduce el número de lecturas
↳ del gen: "))
```

```
## Introduce el número de lecturas del gen:
```

```
lecturas_totales <- as.numeric(readline(prompt = "Introduce el número total de
↳ lecturas de la muestra: "))
```

```
## Introduce el número total de lecturas de la muestra:
```

```
# Calcular el índice de normalización con control de errores
tryCatch({
  if (is.na(lecturas_gen) || is.na(lecturas_totales)) {
    stop("Error: Debes introducir valores numéricos válidos.")
  }

  resultado <- indice_normalizacion(lecturas_gen, lecturas_totales)

  cat("Índice de normalización:", resultado, "\n")
}, error = function(e) {
  cat(e$message, "\n")
})
```

```
## Error: Debes introducir valores numéricos válidos.
```

Con esta modificación, ahora se pueden introducir los valores directamente en la consola, y la función se encarga de validar y calcular el índice de normalización. Así, el mismo planteamiento anterior se adapta a una situación más interactiva, mostrando el resultado o los mensajes de error correspondientes según el caso.

2.3. Ejercicio 3. Análisis farmacocinético con SQL en R

2.3.1. Enunciado

Disponemos del conjunto de datos *Theoph* del paquete **datasets**, que contiene datos de un experimento de farmacocinética de la teofilina en varios pacientes. La teofilina se usa para prevenir y tratar las sibilancias, la falta de aliento y la opresión en el pecho causada por el asma, la bronquitis crónica, el enfisema y otras enfermedades pulmonares.

En el conjunto de datos a trabajar, cada fila representa una medición de concentración (**conc**) en un tiempo (**time**) para un determinado sujeto (**subject**).

Se pide:

- a) Cargar el paquete **datasets** de *RStudio* y el conjunto de datos *Theoph* así como las librerías necesarias, **DBI** y **RSQLite** para ejecutar consultas SQL en R. Mostrad el conjunto de datos *Theoph*
- b) Establecer una conexión a una base de datos SQLite en memoria, cargar el conjunto de datos *Theoph* y escribirlo como una tabla llamada '*Theoph*' en la base de datos.
- c) Realizar una consulta SQL que calcule la concentración media (**conc**) para cada sujeto. El resultado debe mostrar: **Subject, conc_media**.
- d) Realizar una consulta en SQL para obtener el tiempo en el que ocurre la concentración máxima por paciente, es decir, para cada sujeto, obtener el tiempo asociado a su concentración máxima.
- e) Realizar una consulta en SQL que filtre las observaciones en un intervalo de tiempo, para ello, por ejemplo, se pide obtener concentraciones entre 2 y 6 horas.
- f) Realizar una consulta en SQL para mostrar el número de observaciones por paciente. Tras finalizar el listado de consultas, recordad cerrar la conexión.

2.3.2. Resolución

- a) Para cargar el paquete **datasets**, el conjunto de datos *Theoph* y las librerías necesarias para trabajar con bases de datos en R, se puede utilizar el siguiente código:

```
# Cargar el paquete datasets
library(datasets)

# Cargar el conjunto de datos Theoph
data("Theoph")

# Mostrar diez primeras filas del conjunto de datos Theoph
head(Theoph, 10)
```

```
##      Subject    Wt Dose   Time  conc
## 1         1 79.6 4.02  0.00  0.74
## 2         1 79.6 4.02  0.25  2.84
## 3         1 79.6 4.02  0.57  6.57
## 4         1 79.6 4.02  1.12 10.50
## 5         1 79.6 4.02  2.02  9.66
## 6         1 79.6 4.02  3.82  8.58
## 7         1 79.6 4.02  5.10  8.36
## 8         1 79.6 4.02  7.03  7.47
## 9         1 79.6 4.02  9.05  6.89
## 10        1 79.6 4.02 12.12  5.94
```

```
# Cargar las librerías necesarias para SQL
library(DBI)
library(RSQLite)
```

De esta manera, primero se dispone de los datos y después del entorno necesario para consultar esa información mediante SQL.

- b) Para establecer una conexión a una base de datos SQLite en memoria y cargar **Theoph** como tabla, se puede utilizar el siguiente código:

```
# Establecer conexión a una base de datos SQLite en memoria
con <- dbConnect(RSQLite::SQLite(), ":memory:")

# Convertir Theoph a data.frame (IMPORTANTE)
Theoph_df <- as.data.frame(Theoph)

# Cargar el conjunto de datos como tabla en la base de datos
dbWriteTable(con, "Theoph", Theoph_df)

# Verificar que la tabla se ha creado correctamente
tablas <- dbListTables(con)
print(tablas)
```

```
## [1] "Theoph"
```

Al tratarse de una base de datos en memoria, el proceso resulta cómodo para realizar consultas sin necesidad de crear archivos adicionales.

- c) Para calcular la concentración media por sujeto mediante una consulta SQL, se puede utilizar el siguiente código:

```
# Consulta SQL para calcular la concentración media por sujeto
query_media <- "
SELECT Subject, AVG(conc) AS conc_media
FROM Theoph
GROUP BY Subject
"
resultado_media <- dbGetQuery(con, query_media)
print(resultado_media)
```

```
##      Subject conc_media
## 1         1    6.439091
## 2        10    5.930909
## 3        11    4.510909
## 4        12    5.410000
## 5         2    4.823636
## 6         3    5.086364
## 7         4    4.940000
## 8         5    5.782727
## 9         6    3.525455
## 10        7    3.910909
```

```
## 11      8    4.271818
## 12      9    4.893636
```

La consulta utiliza la función agregada **AVG(conc)** para calcular la concentración media de teofilina de cada paciente. La cláusula **GROUP BY Subject** agrupa todas las observaciones pertenecientes a un mismo sujeto, de modo que se obtiene una única media por paciente. El resultado permite comparar de forma resumida el nivel medio de concentración entre sujetos a lo largo del estudio.

- d) Para obtener el tiempo en el que ocurre la concentración máxima por paciente, se puede utilizar la siguiente consulta SQL:

```
# Consulta SQL para obtener el tiempo de concentración máxima por paciente
query_max <- "
SELECT t1.Subject, t1.Time, t1.conc
FROM Theoph t1
WHERE t1.conc = (
  SELECT MAX(t2.conc)
  FROM Theoph t2
  WHERE t2.Subject = t1.Subject
)
"
resultado_max <- dbGetQuery(con, query_max)
print(resultado_max)
```

```
##      Subject Time  conc
## 1         1 1.12 10.50
## 2         2 1.92  8.33
## 3         3 1.02  8.20
## 4         4 1.07  8.60
## 5         5 1.00 11.40
## 6         6 1.15  6.44
## 7         7 3.48  7.09
## 8         8 2.02  7.56
## 9         9 0.63  9.03
## 10        10 3.55 10.21
## 11        11 0.98  8.00
## 12        12 3.52  9.75
```

En este caso, se utiliza una subconsulta correlacionada. Para cada fila de la tabla Theoph, la subconsulta obtiene el valor máximo de concentración (**MAX(conc)**) del mismo sujeto. Después, la consulta principal selecciona únicamente aquellas filas cuya concentración coincide con ese máximo. De este modo, no solo se identifica la concentración máxima de cada paciente, sino también el tiempo exacto en que se alcanza.

- e) Para filtrar las observaciones en un intervalo de tiempo (entre 2 y 6 horas), se puede utilizar la siguiente consulta SQL:

```
# Consulta SQL para filtrar observaciones entre 2 y 6 horas
query_intervalo <- "
SELECT *
FROM Theoph
WHERE Time BETWEEN 2 AND 6
"
resultado_intervalo <- dbGetQuery(con, query_intervalo)
print(resultado_intervalo)
```

```
##      Subject    Wt Dose Time  conc
## 1         1 79.6 4.02 2.02  9.66
## 2         1 79.6 4.02 3.82  8.58
## 3         1 79.6 4.02 5.10  8.36
## 4         2 72.4 4.40 3.50  6.85
## 5         2 72.4 4.40 5.02  6.08
## 6         3 70.5 4.53 2.02  7.80
## 7         3 70.5 4.53 3.62  7.50
## 8         3 70.5 4.53 5.08  6.20
## 9         4 72.7 4.40 2.13  8.38
## 10        4 72.7 4.40 3.50  7.54
## 11        4 72.7 4.40 5.02  6.88
## 12        5 54.6 5.86 2.02  9.33
## 13        5 54.6 5.86 3.50  8.74
## 14        5 54.6 5.86 5.02  7.56
## 15        6 80.0 4.00 2.03  6.32
## 16        6 80.0 4.00 3.57  5.53
## 17        6 80.0 4.00 5.00  4.94
## 18        7 64.6 4.95 2.02  6.58
## 19        7 64.6 4.95 3.48  7.09
## 20        7 64.6 4.95 5.00  6.66
## 21        8 70.5 4.53 2.02  7.56
## 22        8 70.5 4.53 3.53  6.59
## 23        8 70.5 4.53 5.05  5.88
## 24        9 86.4 3.10 2.02  6.33
## 25        9 86.4 3.10 3.53  5.66
## 26        9 86.4 3.10 5.02  5.67
## 27       10 58.2 5.50 2.05  7.83
## 28       10 58.2 5.50 3.55 10.21
## 29       10 58.2 5.50 5.05  9.18
## 30       11 65.0 4.92 3.60  5.87
## 31       11 65.0 4.92 5.02  5.22
## 32       12 60.5 5.30 2.00  9.72
## 33       12 60.5 5.30 3.52  9.75
## 34       12 60.5 5.30 5.07  8.57
```

Esta consulta permite seleccionar únicamente las mediciones comprendidas en ese rango temporal, lo que facilita centrarse en una franja concreta del estudio sin modificar el conjunto de

datos original. La consulta emplea la cláusula **WHERE** junto con la condición **BETWEEN 2 AND 6** para filtrar únicamente las observaciones registradas dentro de ese intervalo temporal. Este tipo de filtrado permite centrar el análisis en una ventana concreta del experimento, lo que puede ser útil para estudiar la evolución de la concentración en una fase determinada tras la administración del fármaco.

- f) Para mostrar el número de observaciones por paciente, se puede utilizar la siguiente consulta SQL:

```
# Consulta SQL para contar el número de observaciones por paciente
query_observaciones <- "
SELECT Subject, COUNT(*) AS num_observaciones
FROM Theoph
GROUP BY Subject
"
resultado_observaciones <- dbGetQuery(con, query_observaciones)
print(resultado_observaciones)
```

##	Subject	num_observaciones
## 1	1	11
## 2	10	11
## 3	11	11
## 4	12	11
## 5	2	11
## 6	3	11
## 7	4	11
## 8	5	11
## 9	6	11
## 10	7	11
## 11	8	11
## 12	9	11

```
# Cerrar la conexión
dbDisconnect(con)
```

En esta consulta se utiliza **COUNT(*)** para contabilizar cuántas mediciones hay registradas para cada sujeto. La cláusula **GROUP BY Subject** agrupa las filas por paciente y permite obtener el número total de observaciones asociadas a cada uno. Este resultado sirve para comprobar si todos los pacientes tienen el mismo número de registros y verificar así la consistencia del conjunto de datos.

3. Sección 2: Probabilidad y simulaciones

3.1. Ejercicio 4. Tiempo de eliminación de un fármaco en sangre

3.1.1. Enunciado

El objetivo de este ejercicio será estudiar el tiempo que tarda un fármaco en reducir su concentración en sangre tras su administración.

Supongamos que el tiempo, en horas, sigue una distribución log-normal y, en condiciones normales, la media meanlog es de 1.0 y la desviación típica sdlog de 0.25.

Se ha detectado que los datos de dichos parámetros en pacientes con insuficiencia renal, serían:

- $\text{meanlog} = 1.4$

- $\text{sdlog} = 0.35$

Nota.- La distribución log-normal es más adecuada para tratar los tiempos biológicos; crecimiento bacteriano,...

Se pide:

- Representar gráficamente las densidades de ambas distribuciones. Comentar las diferencias observadas.
- Calcular la probabilidad de que el fármaco se elimine en menos de 2 horas y en más de 6 horas, en ambos casos. Interpretar los resultados.
- Realizar 10.000 simulaciones en cada escenario y estimar media, varianza y percentiles 25 %, 50 % y 75 %. Comparar con valores teóricos de la media y la varianza teórica de la log-normal.
- Realizar una breve reflexión del estudio realizado a partir de los resultados anteriores.

3.1.2. Resolución

- Para representar gráficamente las densidades de ambas distribuciones log-normales, se puede utilizar el siguiente código. La representación conjunta permite comparar de forma visual cómo cambia el tiempo de eliminación del fármaco entre ambos escenarios.

```
# Parámetros para la distribución log-normal normal
meanlog_normal <- 1.0
sdlog_normal <- 0.25

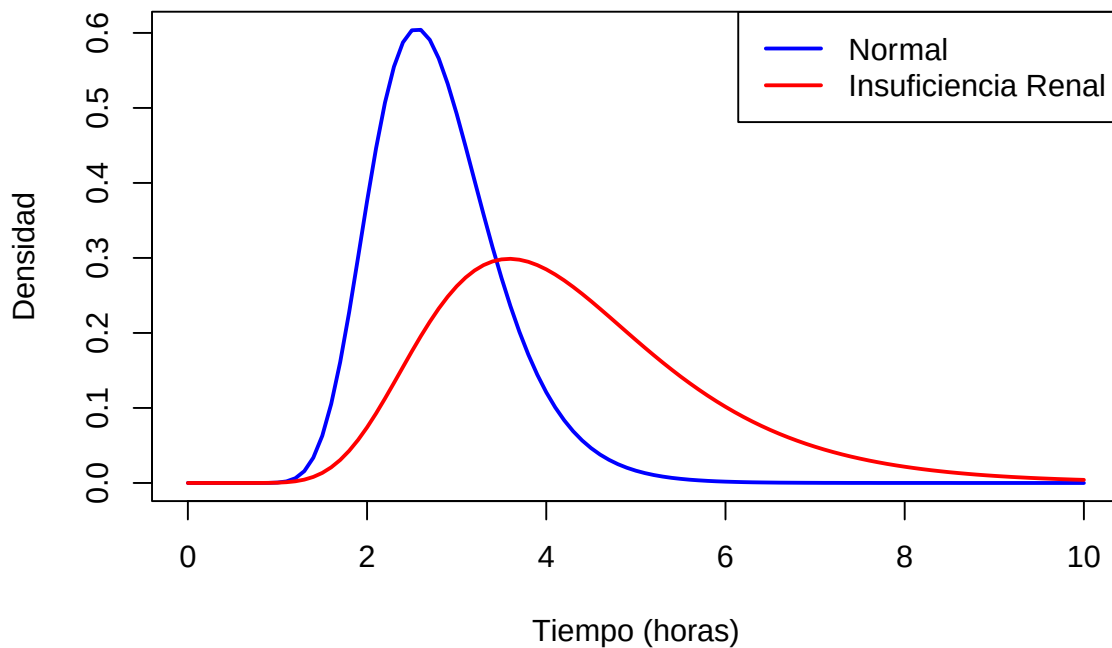
# Parámetros para la distribución log-normal insuficiencia renal
meanlog_insuficiencia <- 1.4
sdlog_insuficiencia <- 0.35
```

```
# Crear una secuencia de tiempo para evaluar las densidades
tiempo <- seq(0, 10, by = 0.1)

# Calcular las densidades para ambas distribuciones
densidad_normal <- dlnorm(tiempo, meanlog = meanlog_normal, sdlog =
  ↪ sdlog_normal)
densidad_insuficiencia <- dlnorm(tiempo, meanlog = meanlog_insuficiencia,
  ↪ sdlog = sdlog_insuficiencia)

# Graficar las densidades
plot(tiempo, densidad_normal, type = "l", col = "blue", lwd = 2,
      xlab = "Tiempo (horas)", ylab = "Densidad",
      main = "Densidad de la distribución log-normal")
lines(tiempo, densidad_insuficiencia, col = "red", lwd = 2)
legend("topright", legend = c("Normal", "Insuficiencia Renal"), col =
  ↪ c("blue", "red"), lwd = 2)
```

Densidad de la distribución log-normal



En la gráfica se observa que la distribución correspondiente a pacientes con insuficiencia renal está desplazada hacia la derecha respecto a la distribución normal, lo que indica que el tiempo de eliminación del fármaco es mayor en estos pacientes. Además, la curva es más ancha, lo que refleja una mayor variabilidad en los tiempos.

Esto se debe a que los parámetros de la distribución (meanlog y sdlog) son más altos en el caso de insuficiencia renal, lo que provoca tanto un aumento en la media como en la dispersión. En términos prácticos, esto sugiere que el fármaco permanece más tiempo en el organismo en

pacientes con insuficiencia renal, lo que podría ser relevante a la hora de ajustar la dosis o la frecuencia de administración.

- b) Para calcular la probabilidad de que el fármaco se elimine en menos de 2 horas y en más de 6 horas en ambas distribuciones, se puede utilizar el siguiente código:

```
# Probabilidad de eliminación en menos de 2 horas
prob_menor_2_normal <- plnorm(2, meanlog = meanlog_normal, sdlog =
  ↪ sdlog_normal)
prob_menor_2_insuficiencia <- plnorm(2, meanlog
  = meanlog_insuficiencia, sdlog = sdlog_insuficiencia)

# Probabilidad de eliminación en más de 6 horas
prob_mayor_6_normal <- 1 - plnorm(6, meanlog
  = meanlog_normal, sdlog = sdlog_normal)
prob_mayor_6_insuficiencia <- 1 - plnorm(6
  , meanlog = meanlog_insuficiencia, sdlog = sdlog_insuficiencia)

# Mostrar resultados
cat("Probabilidad de eliminación en menos de 2 horas:\n")
```

```
## Probabilidad de eliminación en menos de 2 horas:
```

```
cat("Normal:", prob_menor_2_normal, "\n")
```

```
## Normal: 0.109834
```

```
cat("Insuficiencia Renal:", prob_menor_2_insuficiencia, "\n\n")
```

```
## Insuficiencia Renal: 0.02171351
```

```
cat("Probabilidad de eliminación en más de 6 horas:\n")
```

```
## Probabilidad de eliminación en más de 6 horas:
```

```
cat("Normal:", prob_mayor_6_normal, "\n")
```

```
## Normal: 0.0007700013
```

```
cat("Insuficiencia Renal:", prob_mayor_6_insuficiencia, "\n")
```

```
## Insuficiencia Renal: 0.1315034
```

Los resultados muestran que la probabilidad de que el fármaco se elimine en menos de 2 horas es menor en pacientes con insuficiencia renal. En cambio, la probabilidad de que permanezca más de 6 horas en el organismo es claramente mayor en este grupo. Esto sugiere que la eliminación es más lenta en pacientes con insuficiencia renal, lo que puede tener implicaciones en el ajuste de la dosis.

- c) Para realizar 10.000 simulaciones en cada escenario y estimar la media, la varianza y los percentiles, se puede utilizar el siguiente código:

```
# Simulaciones para la distribución normal
simulaciones_normal <- rlnorm(10000, meanlog = meanlog_normal,
                              sdlog = sdlog_normal)
media_normal <- mean(simulaciones_normal)
varianza_normal <- var(simulaciones_normal)
percentiles_normal <- quantile(simulaciones_normal,
                               probs = c(0.25, 0.5, 0.75))

# Simulaciones para la distribución insuficiencia renal
simulaciones_insuficiencia <- rlnorm(10000,
                                     meanlog = meanlog_insuficiencia,
                                     sdlog = sdlog_insuficiencia)
media_insuficiencia <- mean(simulaciones_insuficiencia)
varianza_insuficiencia <- var(simulaciones_insuficiencia)
percentiles_insuficiencia <- quantile(simulaciones_insuficiencia,
                                      probs = c(0.25, 0.5, 0.75))

# Mostrar resultados
cat("Resultados para la distribución normal:\n")
```

```
## Resultados para la distribución normal:
```

```
cat("Media:", media_normal, "\n")
```

```
## Media: 2.805508
```

```
cat("Varianza:", varianza_normal, "\n")
```

```
## Varianza: 0.5091454
```

```
cat("Percentiles (25%, 50%, 75%):", percentiles_normal, "\n\n")
```

```
## Percentiles (25%, 50%, 75%): 2.294067 2.71332 3.219823
```

```
cat("Resultados para la distribución insuficiencia renal:\n")
```

```
## Resultados para la distribución insuficiencia renal:
```

```
cat("Media:", media_insuficiencia, "\n")
```

```
## Media: 4.317505
```

```
cat("Varianza:", varianza_insuficiencia, "\n")
```

```
## Varianza: 2.419274
```

```
cat("Percentiles (25%, 50%, 75%):", percentiles_insuficiencia, "\n")
```

```
## Percentiles (25%, 50%, 75%): 3.223896 4.072857 5.123474
```

Para complementar los resultados obtenidos mediante simulación, se han calculado los valores teóricos de la media y la varianza de la distribución log-normal a partir de sus parámetros (meanlog y sdlog). Estos valores permiten disponer de una referencia con la que comparar los resultados simulados:

```
# Valores teóricos de la distribución log-normal
```

```
# Escenario normal
```

```
media_teorica_normal <- exp(meanlog_normal + (sdlog_normal^2)/2)
```

```
var_teorica_normal <- (exp(sdlog_normal^2) - 1) *  
                      exp(2*meanlog_normal + sdlog_normal^2)
```

```
# Escenario insuficiencia renal
```

```
media_teorica_insuf <- exp(meanlog_insuficiencia + (sdlog_insuficiencia^2)/2)
```

```
var_teorica_insuf <- (exp(sdlog_insuficiencia^2) - 1) *  
                    exp(2*meanlog_insuficiencia + sdlog_insuficiencia^2)
```

```
# Mostrar resultados
```

```
cat("Valores teóricos - Normal:\n")
```

```
## Valores teóricos - Normal:
```

```
cat("Media:", media_teorica_normal, "\n")
```

```
## Media: 2.804569
```

```
cat("Varianza:", var_teorica_normal, "\n\n")
```

```
## Varianza: 0.5072882
```

```
cat("Valores teóricos - Insuficiencia renal:\n")
```

```
## Valores teóricos - Insuficiencia renal:
```

```
cat("Media:", media_teorica_insuf, "\n")
```

```
## Media: 4.311345
```

```
cat("Varianza:", var_teorica_insuf, "\n")
```

```
## Varianza: 2.422333
```

Al comparar los resultados de las simulaciones con los valores teóricos, se observa que el ajuste es bastante bueno en ambos escenarios. En el caso normal, la media y la varianza simuladas son muy próximas a los valores teóricos, y lo mismo ocurre en el caso de insuficiencia renal.

Las pequeñas diferencias observadas se deben al carácter aleatorio del proceso de simulación. Por tanto, los resultados reproducen de manera adecuada el comportamiento esperado de la distribución, confirmando así que el escenario de insuficiencia renal presenta una media más alta y una mayor variabilidad.

- d) A modo de reflexión, el estudio muestra que la insuficiencia renal tiene un impacto importante en el tiempo de eliminación del fármaco, tal como se aprecia en las diferencias entre ambas distribuciones log-normales. En concreto, se observa que los pacientes con insuficiencia renal presentan tiempos de eliminación más elevados y una mayor variabilidad, lo que implica que el fármaco puede permanecer más tiempo en el organismo.

Esto tiene implicaciones relevantes a nivel clínico, ya que podría ser necesario ajustar la dosificación o la frecuencia de administración para evitar acumulación del fármaco. Además, las simulaciones realizadas permiten comprobar que los valores estimados (media, varianza y percentiles) son coherentes con los valores teóricos, lo que refuerza la validez del modelo utilizado.

En conjunto, el análisis permite entender mejor cómo cambian las propiedades farmacocinéticas del fármaco en función del estado fisiológico del paciente.

3.2. Ejercicio 5. Respuesta a un tratamiento analgésico en dolor crónico.

3.2.1. Enunciado

Se requiere evaluar la eficacia de un nuevo tratamiento analgésico en pacientes con dolor crónico del cual sabemos que no todos los pacientes responden igual.

A partir de estudios previos el 25 % de los pacientes responden al tratamiento es decir experimentan una reducción significativa del dolor En cambio el 75 % restante no responden al tratamiento.

En pacientes que responden al tratamiento la probabilidad de manifestar mejoría clínica tras una semana es del 80 % En el resto de pacientes la probabilidad de manifestar mejoría efecto placebo o variabilidad es del 20 %

- Si seleccionamos 120 pacientes tratados ¿cuál es la probabilidad de que al menos 50 reporten mejoría Calcularlo de forma exacta y mediante simulación.
- Si seleccionamos 30 pacientes con respuesta al tratamiento ¿cuál es la probabilidad de que al menos 25 mejoren Y si seleccionamos 80 pacientes que no responden al tratamiento ¿cuál es la probabilidad de que como máximo 25 mejoren.
- Representar la distribución del número de pacientes que mejoran en ambos grupos y comentar los resultados.
- Realizar una breve reflexión del estudio realizado a partir de los resultados anteriores.

3.2.2. Resolución

- Para calcular la probabilidad de que al menos 50 pacientes reporten mejoría entre los 120 tratados, se puede utilizar la distribución binomial, ya que cada paciente puede considerarse un ensayo con dos posibles resultados: mejorar o no mejorar. La probabilidad de éxito para cada paciente es:

- Para pacientes que responden al tratamiento: $0.25 * 0.80 = 0.20$
- Para pacientes que no responden al tratamiento: $0.75 * 0.20 = 0.15$

La probabilidad total de mejoría por paciente es la suma de ambas probabilidades. Con ese valor ya es posible calcular tanto la probabilidad exacta como una estimación por simulación:

```
prob_mejoria <- 0.25 * 0.80 + 0.75 * 0.20
```

```
# Calcular la probabilidad de que al menos 50 pacientes reporten mejoría
prob_al_menos_50 <- 1 - pbinom(49, size = 120, prob = prob_mejoria)
cat("Probabilidad de que al menos 50 pacientes reporten mejoría (exacta):",
    ↪ prob_al_menos_50, "\n")
```

```
## Probabilidad de que al menos 50 pacientes reporten mejoría (exacta): 0.07682402
```

```
# Simulación
set.seed(123) # Para reproducibilidad
simulaciones <- rbinom(10000, size = 120, prob = prob_mejoria)
prob_simulada <- mean(simulaciones >= 50)
cat("Probabilidad de que al menos 50 pacientes reporten mejoría
    ↪ (simulación):", prob_simulada, "\n")
```

```
## Probabilidad de que al menos 50 pacientes reporten mejoría (simulación): 0.0771
```

- b) Para calcular la probabilidad de que al menos 25 de los 30 pacientes con respuesta al tratamiento mejoren, se puede utilizar la distribución binomial con probabilidad de éxito 0.80. Del mismo modo, para el grupo sin respuesta se trabaja con la probabilidad correspondiente a ese caso:

```
# Probabilidad de que al menos 25 de los 30 pacientes con respuesta mejoren
prob_al_menos_25_respuesta <- 1 - pbinom(24, size = 30, prob = 0.80)
cat("Probabilidad de que al menos 25 pacientes con respuesta mejoren:",
  ↪ prob_al_menos_25_respuesta, "\n")
```

```
## Probabilidad de que al menos 25 pacientes con respuesta mejoren: 0.4275124
```

```
# Probabilidad de que como máximo 25 de los 80 pacientes que no responden
  ↪ mejoren
prob_maximo_25_no_respuesta <- pbinom(25, size = 80, prob = 0.15)
cat("Probabilidad de que como máximo 25 pacientes que no responden mejoren:",
  ↪ prob_maximo_25_no_respuesta, "\n")
```

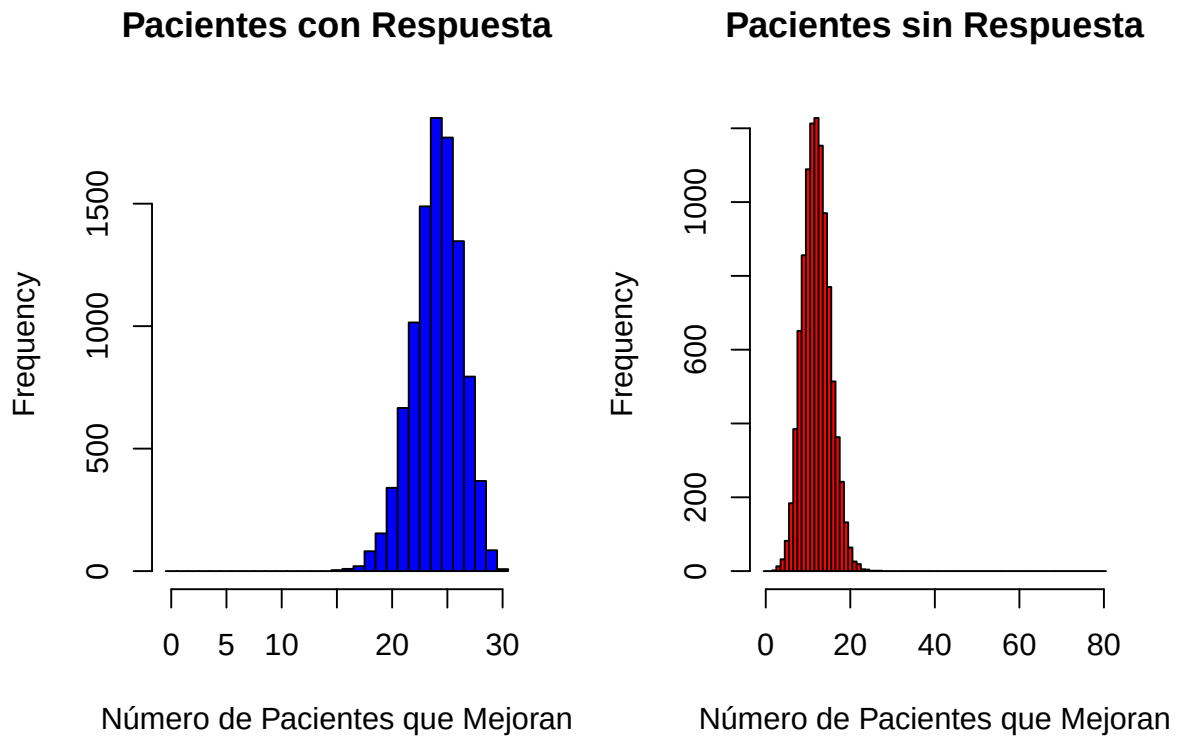
```
## Probabilidad de que como máximo 25 pacientes que no responden mejoren: 0.9999315
```

- c) Para representar la distribución del número de pacientes que mejoran en ambos grupos, se puede utilizar un histograma o una gráfica de barras. A continuación, se muestra cómo hacerlo:

```
# Simulaciones para pacientes con respuesta al tratamiento
simulaciones_respuesta <- rbinom(10000, size = 30, prob = 0.80)

# Simulaciones para pacientes que no responden al tratamiento
simulaciones_no_respuesta <- rbinom(10000, size = 80, prob = 0.15)

# Representar las distribuciones
par(mfrow = c(1, 2)) # Para mostrar ambas gráficas lado a lado
hist(simulaciones_respuesta, breaks = seq(-0.5, 30.5, by = 1), col = "blue",
  ↪ main = "Pacientes con Respuesta", xlab = "Número de Pacientes que
  ↪ Mejoran")
hist(simulaciones_no_respuesta, breaks = seq(-0.5, 80.5, by = 1), col = "red",
  ↪ main = "Pacientes sin Respuesta", xlab = "Número de Pacientes que
  ↪ Mejoran")
```



En la gráfica se puede ver que el número de pacientes que mejoran es claramente mayor en el grupo que responde al tratamiento, ya que la distribución está centrada en valores más altos. En cambio, en el grupo que no responde, la mayoría de los valores se concentran en números bajos.

Esta diferencia se explica por la probabilidad de mejora en cada grupo: en los pacientes que responden es alta (0.80), mientras que en los que no responden es bastante menor (0.15). Esto hace que las distribuciones sean muy diferentes entre sí.

En conjunto, los resultados muestran de forma bastante clara que el tratamiento tiene un efecto notable en los pacientes que responden, mientras que en el resto la mejoría es mucho menos frecuente.

- d) Como reflexión final, el estudio muestra que el nuevo tratamiento analgésico tiene una eficacia variable entre pacientes, con una proporción relevante que no responde. En este caso, la probabilidad de mejoría es mucho mayor en el grupo que sí responde al tratamiento que en el grupo que no responde, tal como se ha visto en los resultados obtenidos.

Esto sugiere que el tratamiento puede ser especialmente útil en un subgrupo concreto de pacientes. Aun así, la existencia de muchos pacientes sin respuesta pone de manifiesto la necesidad de identificar factores predictivos que permitan anticipar qué pacientes se beneficiarán del tratamiento, así como valorar alternativas terapéuticas para aquellos que no muestran mejoría.